

Architectural Blueprint and Landscape Assessment for a Universal MCP Unit Conversion Tool

The integration of physical quantities into computational systems represents one of the most historically perilous challenges in software engineering. While numerical computation is fundamentally deterministic, the attachment of dimensional semantics to scalar or tensor values introduces a multidimensional layer of complexity. Discrepancies in dimensional logic have famously caused catastrophic failures in aerospace engineering, computational physics, and industrial system design. As the industry transitions toward autonomous agentic workflows driven by Large Language Models (LLMs), the unit conversion dilemma has evolved. LLMs possess remarkable linguistic and associative capabilities but are architecturally constrained regarding deterministic mathematical operations. Their probabilistic nature fundamentally conflicts with the strict, non-negotiable algebraic rules governing dimensional analysis.

To bridge this operational gap and provide a foundation for robust AI reasoning, the Model Context Protocol (MCP) has emerged as the definitive standard for integrating external deterministic tooling with probabilistic AI agents. This report serves as a comprehensive landscape assessment and foundational architectural guide to inform a Product Requirements Document (PRD). While formulating the requirements for Google Antigravity, this server must be architected as a universally applicable utility for *any* agentic workflow, capable of shielding varied AI ecosystems from dimensional hallucinations.

The Epistemological Crisis of AI Dimensional Mathematics

The driving motivation for engineering an external, deterministically compiled MCP unit conversion tool is the fundamental inability of Large Language Models to perform reliable dimensional mathematics natively. LLMs are autoregressive token predictors; they optimize for linguistic plausibility and sequence probability rather than absolute mathematical truth.¹ When evaluating scientific or engineering workflows, this probabilistic architecture introduces unacceptable margins of error.

Recent benchmarking of frontier models on scientific reasoning and physical intuition reveals severe, systemic limitations. Platforms such as the Hughes Hallucination Evaluation Model (HHEM) demonstrate that leading models exhibit baseline hallucination rates between 1.8% and 5.3% on standard factual tasks, with parameters varying wildly based on task complexity.² Furthermore, specialized evaluations like the ARC-Challenge (AI2 Reasoning Challenge) indicate that when models are tasked with commonsense physical reasoning and multi-step inference, hallucinations arise from subtle violations of implicit physical laws that are difficult to detect without deep semantic parsing.³

The challenge of LLM hallucination in the domain of dimensional analysis can be further

deconstructed using the HalluLens taxonomy, which distinguishes between extrinsic and intrinsic hallucinations.⁴ Intrinsic hallucinations occur when the model contradicts the specified units within the user's prompt, while extrinsic hallucinations occur when the model fabricates a completely false conversion rate or physical constant that deviates from real-world scientific consensus.⁴ In the context of unit conversion, both forms of hallucination are catastrophic. More insidiously, models are afflicted by a phenomenon known as "High-Certainty Hallucinations." Empirical studies utilizing knowledge detection and uncertainty measurement techniques have demonstrated that models frequently generate dimensionally nonsensical outputs with absolute statistical confidence, even when the foundational knowledge of the correct conversion rate or physical constant resides dormant within their parameterized weights.⁶ An LLM may correctly recite the formula $F = ma$ but simultaneously fail to recognize that supplying mass in grams and acceleration in m/s^2 yields an invalid Newton output unless a 10^{-3} scaling factor is applied. Because the probabilistic token generation process lacks an inherent error-checking mechanism for algebraic consistency¹, the only mathematically sound solution is to offload dimensional computations to an external, strictly typed computational module via the Model Context Protocol.

Benchmark Model	Hallucination Rate	Factual Consistency Rate	Average Summary Length
antgroup/finix_s1_32b	1.8 %	98.2 %	172.4 words
openai/gpt-5.4-nano	3.1 %	96.9 %	144.4 words
google/gemini-2.5-flash-lite	3.3 %	96.7 %	95.7 words
meta-llama/Llama-3.3-70B	4.1 %	95.9 %	64.6 words
mistralai/mistral-large-2411	4.5 %	95.5 %	85.0 words
amazon/nova-pro-v1:0	5.1 %	94.9 %	66.2 words

Data reflecting hallucination rates across various frontier models, underscoring the baseline risk inherent in unmediated LLM outputs.²

Deconstructing Developer Pain Points in Traditional Unit Conversions

The development of a robust unit conversion system requires navigating a minefield of computational anomalies. The historical developer pain points stem from the fundamental mismatch between how computer hardware processes numerical values and how physical reality dictates dimensional relationships. Addressing these pain points is essential for formulating the Google Antigravity PRD.

Precision Dilemmas and Round-Tripping Errors

At the foundation of computational unit conversion lies the severe limitation of standard floating-point arithmetic. Standard computing environments rely heavily on the IEEE 754 floating-point standard, which approximates real continuous numbers using discrete binary fractions. This approximation guarantees precision loss, which becomes painfully evident during multi-step or reciprocal unit conversions.

A ubiquitous developer pain point is the failure of round-tripping—the inability of a scalar value to return to its exact original state after a series of reciprocal dimensional transformations. For

instance, computing the mathematical expression $\frac{1}{98} \times 98$ using standard floating-point logic frequently yields an approximation such as 0.9999999999999999 rather than the

exact integer 1.⁷ When an autonomous agent executes a complex workflow requiring a conversion from meters to miles, from miles to furlongs, and subsequently back to meters, the accumulated floating-point drift can corrupt downstream condition checks, equality assertions, and database integrity.

To circumvent this precision degradation, sophisticated mathematical ecosystems employ rational number representations and arbitrary-precision arithmetic. Libraries such as Fraction.js and Math.js in the JavaScript ecosystem⁷, alongside the fractions module in the Python standard library⁹, manage values as explicit numerator-denominator pairs. Fraction.js, for example, utilizes a native BigInt representation for these pairs, ensuring zero precision loss during intermediate algebraic steps.⁷

Within the Python scientific computing ecosystem, the SymPy library elevates this by treating mathematical operations as symbolic expressions. SymPy heavily leverages rational arithmetic; by explicitly processing numerical inputs via SymPy's sympify functionality (e.g., bypassing standard math division by using sympy.prod and sympy.gcd), computations actively avoid standard float coercion.¹⁰ This preserves the exact mathematical relationship infinitely until a final numerical evaluation (evalf) is explicitly requested by the user or system.¹⁰ Any MCP tooling designed for LLM interactions must natively incorporate rational arithmetic to ensure

deterministic round-tripping.

The $O(n^2)$ Code Complexity Dilemma

A naive, rudimentary approach to unit conversion logic requires writing an explicit conversion function or multiplying factor for every possible pair of units. For a system supporting N units, this paradigm requires $O(N^2)$ discrete conversion pathways, leading to an inherently unmaintainable and bloated codebase.

Modern standardized systems resolve this by employing a hub-and-spoke dimensional matrix. This approach designates a baseline foundational unit (typically derived from the International System of Units, SI) for each physical dimension (e.g., meters for length, kilograms for mass). Every alternative unit is defined exclusively by its specific scaling factor relative to this baseline

unit, thereby reducing the system's algorithmic complexity to $O(N)$ definitions.¹³ To convert from Unit A to Unit B, the system implicitly scales Unit A to the common base unit, and subsequently scales from the base unit to Unit B.

The Python library Pint represents the pinnacle of minimizing code complexity through intelligent parsing. Pint utilizes a standalone, text-based unit definition file that allows for the dynamic parsing of prefixes and pluralizations without requiring explicit combinatorial hardcoding.¹⁴ Because the prefix "kilo" and the base unit "meter" are defined independently within Pint's configuration, the library autonomously understands and evaluates "kilometers".¹⁴ This architectural decision drastically reduces the unit definition footprint compared to older packages, while enabling vast flexibility for an AI agent that might format unit strings unpredictably.

Strong Typing vs. Runtime Erasure

A fundamental philosophical divide in the architecture of unit libraries is the choice between strictly enforced compile-time dimensional checking and dynamic runtime evaluation. In statically typed systems programming languages, the ultimate objective is the creation of "zero-cost abstractions," where all dimensional verification occurs entirely during the compilation phase, imposing absolutely zero performance overhead at runtime. The Rust ecosystem exemplifies this through the uom (Units of Measurement) crate. By leveraging Rust's rigorous trait system and the typenum crate, uom evaluates type-level numbers at compile time, achieving automatic, type-safe dimensional analysis that mathematically guarantees that a system cannot accidentally compile code that adds thrust forces to temperature gradients.¹⁵ However, the pursuit of absolute compile-time safety introduces severe limitations for dynamic environments, including AI agent interactions via MCP protocols. The functional programming language F# highlights this constraint. F# famously incorporates Units of Measure directly into its compiler type system, allowing developers to strongly type variables such as `float<kg>` or `float<m/s>`.¹⁹ Yet, to comply with the execution constraints of the .NET Common Language Runtime (CLR), these dimensional tags are completely erased during the compilation process.¹⁹

Consequently, any attempt to implement runtime logic that depends on verifying a value's unit is impossible in F#; runtime reflection cannot determine whether a raw floating-point value in memory represents a distance or a velocity.¹⁹

Because an LLM utilizing an MCP tool operates entirely asynchronously and at runtime, relying on compile-time erased types is fundamentally unviable. The MCP tool must employ explicit runtime dimensional tracking, retaining and carrying metadata tags alongside the numerical payload throughout execution. This makes the dynamic evaluation paradigm found in Python ecosystems (e.g., Pint, Astropy) far superior for LLM agent integration than strictly compiled systems, despite the theoretical performance penalty.

Unit Consistency During Algebraic Operations and Integer Overflow

When physical quantities are mathematically combined, their underlying dimensions must

interact algebraically. If an AI agent attempts to divide a distance (m) by a time (s), the resulting unit must automatically resolve to a velocity (m/s). Engineering an API that seamlessly computes these dimensional combinations without hardcoding every permutation is exceptionally difficult.

The historical evolution of unit libraries in the C++ ecosystem highlights the immense difficulty of achieving algebraic consistency. The legacy `boost::units` library, bound to the constraints of the C++98 standard, achieved generic dimensionality via complex template metaprogramming.²⁴ However, this resulted in an infamously poor developer experience, characterized by inscrutable compiler error outputs featuring deeply nested, nearly unreadable typenames.²⁴ Modern C++ alternatives have sought to rectify this. The `mp-units` library requires a minimum of C++20 to leverage ultra-modern language features such as concepts and non-type template parameters (NTPs), heavily improving the fluidity of composability and allowing highly readable compiler output.²⁴ Alternatively, the `Au` library utilizes C++14 to strike a balance between modern conveniences and broad backward compatibility, providing 100% unit-safe interfaces.²⁴

Yet, even within these modern C++ libraries, a dangerous developer pain point persists: the risk of silent integer overflow during implicit algebraic scaling. As noted by the architects of the `mp-units` ecosystem, overflow is an insidious problem that easily hides behind the abstraction layers of a units library.²⁷ When a system evaluates an algebraic expression such as $1 * m + 1 * ft$, the library must automatically scale both operands to a common baseline unit before the addition can be completed.²⁷ If the underlying storage architecture utilizes integer representations rather than floating-point arrays, the intermediate multiplication by large scaling factors can silently overflow the integer's maximum capacity.²⁷ Because this occurs without throwing an exception, it produces catastrophic garbage values that propagate undetected through downstream calculations. While the `Au` library attempts to mitigate this vulnerability via a unique adaptive "safety surface" that gauges overflow risk based on conversion magnitude, it remains a critical vulnerability in static systems.²⁴ An MCP server must therefore strictly govern the data types used during algebraic resolution, preferring

arbitrary-precision math over native integers.

Library	Language	Strong Typing Paradigm	Primary Trade-off	Affine Space Support
Boost.Units	C++ (C++98)	Compile-time Templates	Unreadable error logs	Basic absolute wrapper
mp-units	C++ (C++20)	C++20 Concepts & NTPPs	Compilation speed	Advanced origin points
Au	C++ (C++14)	Compile-time Safety Surface	Complex generic dimensions	Custom Zero type
uom	Rust	Zero-cost trait analysis	Steep learning curve	N/A
Pint	Python	Runtime object verification	Performance overhead	Explicit delta units
F# Core	F#	Compile-time syntax	Runtime erasure (Lost metadata)	N/A

Comparison of primary unit conversion libraries detailing their approach to dimensional typing and notable trade-offs.¹⁴

Critical Blind Spots and Systematic Failures in Edge Cases

While standard multiplicative unit conversions (e.g., converting meters to kilometers or watts to kilowatts) are managed adequately by basic scripting logic, the computational landscape fractures severely when confronted with non-linear, affine, or context-dependent physical realities. It is precisely within these edge cases that naive implementations systematically fail, and where a sophisticated MCP tool must establish rigorous safeguards.

Lexical and Semantic Ambiguity in Natural Language

When processing natural language inputs from an LLM, the system must navigate severe textual representation ambiguity. A single unit can be represented in multiple forms (e.g., "inch",

"in.", "in", or "). Attempting to hardcode every variation leads to parser failure. Standards such as the Unified Code for Units of Measure (UCUM) were explicitly developed for machine-to-machine communication to resolve naming conflicts and provide unambiguous mappings between case-sensitive and case-insensitive textual representations. Furthermore, the system must overcome semantic ambiguity, where identical text represents vastly different concepts. For instance, an LLM outputting the string "min" could be referring to a "minute" (a time unit) or a combination of the prefix "milli" and the unit "inch" (milli-inch). To prevent parsing hallucinations, the tool's schema must cleanly enforce strict unit ontologies that map abbreviations to unambiguous mathematical definitions.

Dimensional Devolution and "Quantity Kinds"

A fundamental mathematical property of dimensional analysis is that every physical quantity devolves into some permutation of the seven fundamental SI base dimensions (mass, length, time, temperature, electric current, luminous intensity, and amount of substance). However, this devolution creates a dangerous programmatic blind spot.

Consider Work/Energy and Torque: both of these quantities resolve to the exact same base

dimensional footprint of ML^2T^{-2} . Despite sharing the identical underlying base dimensions, they possess entirely different physical origins; energy is a scalar quantity, whereas torque is a vector quantity resulting from a force applied across a directional radius of curvature. If a conversion system relies solely on dimensional devolution, it will incorrectly validate the substitution of Joules (Energy) for Newton-meters (Torque).

To solve this, advanced systems introduce the concept of "Quantity Kinds" or "Quantity Characters". The QUDT (Quantities, Units, Dimensions and Types) ontology explicitly binds units to specific QuantityKind classifications—such as differentiating between quantitykind:Energy and quantitykind:Torque—acting as a metadata layer that prevents mathematically plausible but physically invalid conversions. Modern programmatic implementations, such as the C++ mp-units library, enforce this via "quantity character" concepts, ensuring that scalar representations cannot be mistakenly applied to vector quantities. A universally applicable MCP server must incorporate this metadata layer to preserve physical truth.

The Affine Space Problem: Non-Multiplicative Temperature Units

The most common, and arguably most destructive, source of systematic failure in computational unit conversion involves thermodynamic temperature scales. Standard physical dimensions operate within a linear vector space where the concept of absolute zero is fixed

and universally shared (e.g., 0 meters represents the exact same physical reality as 0 inches). Temperature scales such as Celsius and Fahrenheit, however, operate within an affine space—they possess entirely different zero reference points.²⁸

Consequently, converting absolute temperatures requires a mathematical operation involving both a multiplicative scaling coefficient and a static additive offset:

$$T_F = T_C \times \frac{9}{5} + 32$$

$$T_K = T_C + 273.15$$

$$T_R = T_F + 459.67$$

The presence of this offset profoundly disrupts basic algebraic principles. In an affine space, the statement $10^\circ C + 10^\circ C = 20^\circ C$ is mathematically undefined and physically nonsensical, because adding two absolute coordinates on an offset scale does not yield a valid sum of energy. Thermal physics mandates a strict programmatic distinction between an *absolute temperature* (a specific coordinate) and a *temperature difference* (a delta or gradient).²⁸

The mathematical formula for converting a delta temperature strictly omits the offset:

$$\Delta T_F = \Delta T_C \times \frac{9}{5}$$

$$\Delta T_K = \Delta T_C$$

If a software system naively applies the absolute conversion formula to a delta value, the output is catastrophically incorrect. For example, a temperature rise of $41^\circ C$ is mathematically equivalent to a rise of $73.8^\circ F$.²⁸ If the system erroneously applies the offset logic ($41 \times \frac{9}{5} + 32$), it calculates $108.8^\circ F$.²⁸ This precise error was documented in the Building Operation software ecosystem, injecting massive logic hallucinations into the software's dataset.²⁸

Advanced numerical libraries handle this discrepancy by explicitly segregating absolute and delta unit typings. The Python library Pint explicitly defines degC and degF as absolute units, and autonomously generates delta_degC and delta_degF counterparts.²⁹ Subtracting two

absolute degC quantities in Pint correctly yields a delta_degC quantity, preserving thermodynamic validity.²⁹ Furthermore, adding a heating rate (e.g., 0.5 kelvin/min) to an absolute starting temperature successfully outputs a new absolute coordinate without throwing an error.²⁹ An MCP agent interface designed for Google Antigravity must expose this semantic distinction strictly, requiring the LLM to specify via the JSON schema whether a queried temperature value represents a static coordinate or a gradient flow.

Logarithmic Constraints: Decibels, Dex, and Magnitudes

Logarithmic units—such as decibels (dB), decibel-milliwatts (dBm), astronomical magnitudes (mag), and dex—represent another complex class of non-linear conversions where traditional arithmetic logic utterly fails. These units represent the logarithm of a ratio relative to a specific reference value, requiring dedicated parsing and evaluation logic.³³ For instance, in radio frequency engineering and optics, the fundamental relationship between absolute power in milliwatts (x) and logarithmic power in dBm (y) is defined as:

$$y \text{ dBm} = 10 \log_{10} \left(\frac{x}{1 \text{ mW}} \right)$$

This definition dictates that 0 dBm equals 1 mW , 10 dBm equals 10 mW , and critically, -10 dBm equals 0.1 mW .³⁴ Converting the result back to the linear domain requires exponential transformation:

$$x \text{ mW} = 10^{\frac{y}{10}}$$

Within the Astropy library (astropy.units), logarithmic units are managed through highly specialized Python classes (Magnitude, Dex, Decibel) that inherit from a central LogQuantity base class, which encapsulates a physical underlying unit.³³ The arithmetic behavior of these instantiated objects is rigidly constrained because standard linear mathematics does not map to logarithmic space. Mathematically, adding two logarithmic quantities implies the multiplication of their underlying physical values. Consequently, Astropy explicitly disables most standard NumPy universal functions (ufuncs) for these units; applying a statistical function such as np.std to a logarithmic array lacks physical meaning and triggers an exception unless explicitly transformed back to linear space first.³³

Furthermore, multiplying a physical scalar quantity by a dimensioned logarithmic unit triggers strict type enforcement errors in Astropy. As documented in community diagnostics, while the algebraic product of a dimensionless quantity and a Decibel triggers a valid logarithmic shift, multiplying a dimensioned Quantity by a DecibelUnit raises an unsupported operand

TypeError.³⁵ Pint handles logarithmic units natively but requires specialized definitional structures to ensure that mixed regular-dB operations (e.g., calculating propagation loss at -0.2 dB/km) evaluate accurately.³⁴ Any MCP tool attempting to service sophisticated electrical engineering, acoustics, or astrophysics LLM queries must natively parse logarithmic syntax and comprehensively isolate its arithmetic evaluation pipeline from standard linear operations to prevent hallucinated physics.

Astropy Logarithmic Class	Primary Domain	Conversion Mechanism	Operational Constraints
astropy.units.Decibel	Signal Processing, RF	Log ratio of powers/amplitudes	Array ufuncs disabled natively
astropy.units.Magnitude	Astrophysics (Flux)	Log ratio of luminosity	Order of operations strictly enforced
astropy.units.Dex	General Astronomy	Base-10 logarithmic shifts	Dimensionless conversion requirements

Breakdown of logarithmic unit isolation in the Astropy ecosystem.³³

Dimensional Inversion: The Mathematics of Fuel Economy

While the vast majority of physical unit conversions strictly preserve the fundamental dimensionality ratio (e.g., converting a velocity in $\frac{km}{h}$ to $\frac{m}{s}$ merely requires a scalar coefficient), certain domain-specific measurements mathematically invert the numerator and denominator depending entirely on the geographic region of usage. The paramount, and most frequently requested, example of this anomaly is fuel economy.³⁸

In the United States and the United Kingdom, vehicular fuel efficiency is universally measured as the distance traveled per unit of volume consumed: Miles Per Gallon (MPG). Conversely, in metric-centric global regions, efficiency is measured as the volume of fuel required to travel a fixed distance: Liters per 100 kilometers (L/100km).³⁹

Converting between these two paradigms requires calculating a strict inverse mathematical relationship, rather than applying a straightforward linear multiplier. The exact algorithmic formula required is:

$$L/100km = \frac{235.214583}{MPG_{US}}$$

This inversion dictates a counterintuitive operational logic: a higher numerical value in the MPG format represents *better* mechanical efficiency, whereas a higher numerical value in the L/100km format represents *worse* mechanical efficiency.⁴⁰ Standard, hardcoded unit

conversion matrices that rely strictly on $y = kx$ multiplication formulas fail completely when attempting to service fuel economy queries. The architectural design of the MCP server must support dimensional parameter inversion natively, scanning the syntax tree for fuel economy variables and applying inverse routing.³⁹

Context-Dependent and Substance-Specific Conversions

Perhaps the most complex computational blind spot in dimensional mathematics is the necessity of environmental or material context. Certain units belong to fundamentally incompatible scientific dimensions, yet they are practically, and frequently, convertible given a specific physical environment or a known chemical substance.

The Spectroscopy Context (Wavelength to Frequency): In standard dimensional logic, attempting to convert a measurement in nanometers (which possesses the dimension of

length, $[L]$) to Hertz (which possesses the dimension of inverse-time, $1/$) is inherently invalid. Attempting this raw operation will appropriately trigger a DimensionalityError in strictly typed libraries like Pint or Astropy.⁴³ However, within the specific scientific context of electromagnetic radiation and spectroscopy, these dimensions are inextricably linked by the speed of light in a

vacuum ($c = \lambda\nu$). The Python library Pint elegantly solves this dilemma via the implementation of programmable "contexts." By executing the command `q.to('Hz', 'spectroscopy')` or utilizing a context manager with `ureg.context('sp')`, the library temporarily alters the underlying dimensional graph, bridging length and inverse-time utilizing the physical

constant c , allowing the evaluation to compile and execute successfully.⁴³

The Culinary and Chemical Context (Volume to Mass): An equally ubiquitous, yet programmatically frustrating, real-world requirement involves converting volumetric

measurements to mass metrics (e.g., translating "2 cups" to "grams"). Because volume ($[L]^3$)

and mass ($[M]$) exist as independent physical dimensions, this conversion fundamentally

requires the specific volumetric density ($\rho = m/V$) of the substance in question.⁴⁵

Commercial APIs, such as the API League Convert Units API and the Alfa Chemistry Density Unit Converter, manage this discrepancy by accepting a specific material, chemical, or food name as an implicit secondary variable.⁴⁶ By querying a backend material database, they

retrieve the specific density coefficient required to bridge the dimensional gap.⁴⁷ An expert-level MCP tool designed for broad AI agent usage must implement an internal substance database—potentially utilizing an auto-generated SQLite vector database similar to open-source drilling community repositories—to act as an autonomous, dynamic context provider for volume-to-mass transformations.⁴⁶

Architectural Ecosystem Analysis

The landscape of dimensional calculation libraries spans multiple programming paradigms, each offering distinct advantages and trade-offs. Formulating the ultimate MCP server for Google Antigravity requires synthesizing the strengths of these disparate ecosystems while discarding their inherent weaknesses.

The Python Ecosystem: Unmatched Flexibility and Dynamic Evaluation

The scientific Python ecosystem overwhelmingly dominates contemporary numerical computing, primarily due to its expressive syntax and tight integration with the NumPy array architecture.

1. **Pint:** As previously noted, Pint operates as an exceptional standalone, zero-dependency library.¹⁴ Its primary architectural triumph is its text-based unit definition registry, which facilitates dynamic parsing without extensive hardcoding. Furthermore, Pint provides seamless, native support for NumPy ndarray objects, allowing array-wide mathematical operations while continuously enforcing dimensional integrity without requiring external monkey-patching.¹⁴
2. **Astropy.units:** Engineered specifically for the extreme rigors of astrophysics, Astropy features highly rigorous handling of complex logarithmic systems and integrates directly with the `astropy.table.Table` structure. This enables multidimensional, unit-aware datasets that far exceed standard DataFrame capabilities.³⁷ However, it is an exceptionally heavyweight computational dependency.
3. **unyt:** Originating from the yt project, unyt provides tightly coupled subclasses of `numpy.ndarray` (`unyt_array` and `unyt_quantity`) designed to represent arrays and scalars with attached metadata.⁵¹ It is highly optimized for performance and integrates intimately with the NumPy ufunc ecosystem. This integration guarantees that algebraic array calculations automatically verify unit consistency prior to execution, preventing costly late-stage crashes.⁵¹
4. **SymPy:** SymPy's dedicated `physics.units` module diverges from the pack by treating units strictly as symbolic mathematical expressions rather than floating-point wrappers.¹³ This fundamental shift allows an MCP tool to not simply calculate numerical values, but to symbolically reduce and factorize complex algebraic expressions (e.g., simplifying a custom derived physics formula algebraically before assigning numerical constants).¹³ Its sophisticated integration of distinct Unit Systems (such as SI versus CGS or Gaussian) handles the complex reality that certain physical constants (like the speed of light)

evaluate to 1 in theoretical natural unit systems, but possess highly specific

dimensioned values in SI environments.¹³

C++, Rust, and Enterprise Java: Strict Compilation Constraints

Systems programming environments approach units with a focus on absolute compilation safety and strict memory layouts.

- **mp-units and Au (C++):** These modern C++ frameworks push the boundaries of template metaprogramming. They excel at enforcing rules at compile-time and handle edge cases like custom generic origins (affine point coordinates) with extreme elegance.²⁴ However, as previously established, their susceptibility to integer overflow and reliance on pre-compiled strict typings render them fundamentally incompatible with the dynamic, string-based prompt structures generated by LLMs.
- **uom (Rust):** By leveraging the typenum crate, Rust's uom achieves zero-cost dimensional analysis that is unparalleled for embedded systems or flight controller software.¹⁵ Yet, it suffers from the same dynamic rigidity as C++.
- **JSR 385 (Java):** In the enterprise Java space, dimensional mathematics is officially governed by JSR 385 (Units of Measurement API 2.2). This specification provides a strictly defined interface layer that completely separates client implementations from core computational logic.⁵⁴ JSR 385 relies heavily on the implementation of a Prefix object acting as a multiplier applied to a base Unit object (e.g., invoking `MetricPrefix.KILO(Units.METRE)`), heavily prioritizing verbose object-oriented design patterns over the symbolic string parsing required for natural language LLM interaction.⁵⁶

Blueprint for the Google Antigravity MCP Unit Server

The Model Context Protocol (MCP) acts as the standardized communication bridge between probabilistic AI agents and deterministic external systems, frequently described as the "USB-C for AI integrations".⁵⁸ To construct an exhaustively complete unit conversion server suitable for integration into a sophisticated environment like Google Antigravity, the system must meticulously orchestrate the protocol's stateful JSON-RPC 2.0 capabilities while incorporating the mathematical safeguards analyzed above.

MCP Communication Architecture and State Management

The MCP ecosystem operates via a strict, multi-tiered client-server hierarchy⁶¹:

1. **MCP Host:** The overarching application environment (e.g., an AI-powered IDE, a financial analysis terminal, or Claude Desktop) encompassing the LLM.
2. **MCP Client:** The internal routing mechanism that translates the LLM's natural language tool-selection intent into protocol-compliant JSON-RPC requests.
3. **MCP Server:** The external computational engine (the proposed Unit Conversion backend) that actively registers its computational capabilities, receives RPC payloads, executes deterministic logic, and streams results back to the agent.⁶²

Communication utilizes standard JSON-RPC formatting, typically operating over bidirectional transports such as standard input/output (stdio) or Server-Sent Events (SSE).⁶³ A tool invocation relies on the `tools/call` method, accompanied by an arguments payload that the LLM

dynamically constructs based on the tool's defined OpenAPI schema.⁶³

Overcoming "Tool Explosion" via Intent-Based Schema Design

A critical architectural failure pattern frequently observed in the development of early-stage MCP interfaces is "tool explosion"—the practice of creating a distinct, atomic protocol tool for every conceivable permutation of a task (e.g., exposing separate `convert_m_to_ft`, `convert_kg_to_lbs`, and `convert_celsius_to_fahrenheit` endpoints).⁶⁵ Exposing hundreds of granular tools floods the LLM's limited context window, degrades its instruction-following capacity, and actively induces intent-matching failures.⁶⁵

Instead, the Google Antigravity MCP server must utilize Intent-Based Grouping, exposing a consolidated, highly expressive tool schema.⁶⁵ The server should expose a singular, primary `dimensional_compute` tool governed by a robust JSON Schema that allows the LLM to pass raw string expressions for internal parsing.

Optimizing the Tool Schema Description

The cognitive interface between the MCP server and the autoregressive LLM is the tool's description field.⁵⁹ To maximize tool-selection accuracy, the schema description must employ:

- **Front-Loaded Intent:** Clearly defining the verb and resource in the opening sequence (e.g., "Computes and converts physical quantities...").
- **Exclusionary Guidance:** Providing explicit negative constraints to prevent hallucinated usage (e.g., "Do not use for raw financial currency conversions. Do not use for standalone basic arithmetic unless physical units are explicitly attached.").⁶⁵
- **Schema Annotations and Few-Shot Examples:** Supplying explicit JSON examples of valid payloads within the description string significantly assists the autoregressive model in predicting the correct syntax structure, dramatically reducing formatting errors.⁶⁶

JSON

[

"name" "dimensional_compute"

"description" "Evaluates algebraic expressions involving physical units and converts to target units. Use this for all metric, imperial, temperature, logarithmic (dB), and substance-specific density calculations. Provide expressions strictly as strings (e.g., '10 m/s * 5 s'). Do not use for basic arithmetic without units."

"inputSchema"

"type" "object"

"properties"

"expression"

```

    "type" "string"
    "description" "The algebraic physics expression to evaluate."
    "target_unit"
    "type" "string"
    "description" "The final unit abbreviation to convert the result into."
    "context"
    "type" "string"
    "enum" "standard" "spectroscopy" "fuel_economy"
    "description" "Select 'spectroscopy' for Hz/nm, or 'fuel_economy' for MPG/L-100km."
    "substance"
    "type" "string"
    "description" "Required ONLY if converting volume to mass (e.g., 'water', 'steel')."
    "required" "expression" "target_unit"

```

Security Topologies and AST-Based Sandboxing

Because the tool is designed to accept complex algebraic strings to handle intricate physical combinations (e.g., $(5 \text{ kg} * 9.8 \text{ m/s}^2) / (2 \text{ m}^2 * 10 \text{ dB})$), the backend must parse and evaluate this input mathematically.⁶⁸ Relying on Python's native `eval()` or `exec()` functions to process LLM-generated string inputs introduces a catastrophic Remote Code Execution (RCE) vulnerability, exposing the underlying host environment to adversarial injection.⁶⁸

A production-grade MCP server (comparable in security to the existing MCP Mathematics open-source implementation) must route all string inputs through an Abstract Syntax Tree (AST) evaluator.⁶⁸ By traversing the generated AST, the server acts as a mathematical sandbox. It explicitly whitelists only safe mathematical operators (+, -, *, /,), immediately strips out any injected system function calls or variable assignments, and passes the sanitized, verified tree structure to the backend compute engine (e.g., Pint or SymPy) for final dimensional resolution.⁶⁸

Leveraging the Full Protocol: Resources and Prompts

The MCP standard extends significantly beyond executable tools to include **Resources** and **Prompts**, which are vital for establishing a comprehensive "one-stop-shop" environment.⁵⁹

- **Resources:** Resources provide structured, readable context data that the LLM can ingest to improve its domain knowledge.⁵⁹ The server should expose a `list_supported_units` resource. This acts as a dynamic, read-only dictionary that the LLM can query into its

context window if it is mathematically unsure of the correct abbreviation format (e.g., whether the backend expects degC, celsius, or C). Additionally, a comprehensive vector database of common substance densities should be exposed as an MCP Resource, providing transparent reference material to prevent the LLM from hallucinating specific gravity values.⁴⁹

- **Prompts:** Prompts provide reusable, pre-configured workflow templates.⁵⁹ The server can offer a prompt titled `verify_dimensional_consistency`. When triggered by the end-user via the host interface, this prompt autonomously instructs the LLM to scan a provided technical document, extract all implicit physical quantities, pass them sequentially through the `dimensional_compute` tool, and generate a formalized validation report highlighting any broken physics equations or hallucinated values within the original text.

Handling Core Pain Points in the Execution Layer

When the MCP Server receives the sanitized and validated JSON payload, the backend engine—architecturally recommended as a hybrid fusion of Python's SymPy for absolute symbolic exactness and Pint for flexible string parsing and context management—must autonomously execute the following deterministic workflow:

1. **Fractional Preservation Protocol:** All scalar numerical values extracted from the AST are immediately cast to rational mathematical fractions (e.g., utilizing `Fraction(numerator, denominator)` or SymPy's `Rational` classes) prior to any conversion factors being applied. This explicitly guarantees zero precision drift and ensures perfect round-tripping capabilities.⁷
2. **Affine Space Detection and Segregation:** The compute engine scans the AST for any temperature units. If an algebraic operation (such as multiplication or division) involves a temperature unit, the engine strictly verifies whether the unit is tagged as an absolute coordinate or a delta unit. If the LLM attempts to multiply an absolute temperature, the engine halts execution and returns a controlled, explanatory MCP JSON-RPC error detailing the thermodynamic violation, thereby prompting the LLM to self-correct its syntax to utilize a delta unit.²⁸
3. **Contextual Routing and Inversion:** If a substance parameter is provided within the JSON payload, the engine dynamically queries its internal SQLite density database and injects a temporary volume/mass equivalence node into the dimensional evaluation graph.⁴³ Concurrently, if the `context="fuel_economy"` flag is triggered, the engine bypasses standard multiplicative matrices, preparing to execute a reciprocal scalar transformation to accurately map MPG to metric standards.³⁹

By strictly offloading the immutable laws of dimensional physics to this deterministically governed, AST-sandboxed external server, software architects can construct AI ecosystems where the LLM is freed to focus entirely on semantic intent and workflow orchestration. The Model Context Protocol, serving as the immutable conduit, guarantees absolute mathematical and physical integrity, permanently eliminating dimensional hallucinations from the agentic

pipeline.

Works cited

1. LLM Hallucination: Understanding AI Text Errors - Label Your Data, accessed May 29, 2026, <https://labelyourdata.com/articles/llm-fine-tuning/llm-hallucination>
2. vectara/hallucination-leaderboard - GitHub, accessed May 29, 2026, <https://github.com/vectara/hallucination-leaderboard>
3. HalluScan: A Systematic Benchmark for Detecting and Mitigating Hallucinations in Instruction-Following LLMs - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2605.02443v2>
4. HalluLens: LLM Hallucination Benchmark - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2504.17550v1>
5. HalluLens: LLM Hallucination Benchmark - ACL Anthology, accessed May 29, 2026, <https://aclanthology.org/2025.acl-long.1176.pdf>
6. Trust Me, I'm Wrong: High-Certainty Hallucinations in LLMs - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2502.12964v1>
7. rawify/Fraction.js: The RAW rational numbers library written in JavaScript - GitHub, accessed May 29, 2026, <https://github.com/infusion/Fraction.js/>
8. Units - Math.js, accessed May 29, 2026, <https://mathjs.org/docs/datatypes/units.html>
9. fractions — Rational numbers — Python 3.14.5 documentation, accessed May 29, 2026, <https://docs.python.org/3/library/fractions.html>
10. Taming math and physics using SymPy - Course Websites, accessed May 29, 2026, https://courses.physics.illinois.edu/tam210/sp2024/sympy_tutorial.pdf
11. Sympy precision using rationals - Stack Overflow, accessed May 29, 2026, <https://stackoverflow.com/questions/72890633/sympy-precision-using-rationals>
12. Numerical Evaluation - SymPy 1.14.0 documentation, accessed May 29, 2026, <https://docs.sympy.org/latest/modules/evalf.html>
13. Unit Systems - SymPy 1.14.0 documentation, accessed May 29, 2026, <https://docs.sympy.org/latest/modules/physics/units/index.html>
14. a Python units library — pint 0.6 documentation - Read the Docs, accessed May 29, 2026, <https://pint.readthedocs.io/en/0.6/>
15. Crate uom - Rust - Docs.rs, accessed May 29, 2026, <https://docs.rs/uom/latest/uom/>
16. uom - crates.io: Rust Package Registry, accessed May 29, 2026, <https://crates.io/crates/uom/dependencies>
17. GitHub - iliekturtles/uom: Units of measurement -- type-safe zero-cost dimensional analysis, accessed May 29, 2026, <https://github.com/iliekturtles/uom>
18. Rust Type System: Making complex things simple. - Stephen Waits, accessed May 29, 2026, <https://swaits.com/beautiful-experience-with-rust-static-type-system/>
19. F Sharp Programming/Units of Measure - Wikibooks, open books for an open world, accessed May 29, 2026, https://en.wikibooks.org/wiki/F_Sharp_Programming/Units_of_Measure
20. Tutorial: Create a Type Provider - F# - Microsoft Learn, accessed May 29, 2026,

<https://learn.microsoft.com/en-us/dotnet/fsharp/tutorials/type-providers/creating-a-type-provider>

21. Make Units of Measure available to more types · Issue #563 · fsharp/fslang-suggestions - GitHub, accessed May 29, 2026, <https://github.com/fsharp/fslang-suggestions/issues/563>
22. Why does F# not allow for automatic unit conversions? : r/fsharp - Reddit, accessed May 29, 2026, https://www.reddit.com/r/fsharp/comments/ikesxu/why_does_f_not_allow_for_automatic_unit/
23. When should I NOT use Units of Measure? - Stack Overflow, accessed May 29, 2026, <https://stackoverflow.com/questions/36586580/when-should-i-not-use-units-of-measure>
24. Comparison of Alternatives - The Au units library, accessed May 29, 2026, <https://aurora-opensource.github.io/au/main/alternatives/>
25. A units library should be specified in terms of concepts · mpusz mp-units · Discussion #194, accessed May 29, 2026, <https://github.com/mpusz/mp-units/discussions/194>
26. The Power of C++ Templates With mp-units: Lessons Learned & a New Library Design - Mateusz Pusz 2023 - YouTube, accessed May 29, 2026, <https://www.youtube.com/watch?v=eUdz0WvOMm0>
27. Preventing Integer Overflow in Physical Computations - mp-units : r/cpp - Reddit, accessed May 29, 2026, https://www.reddit.com/r/cpp/comments/1sqkrhd/preventing_integer_overflow_in_physical/
28. Delta Temperature Conversion - Building Management WebHelp - Schneider Electric, accessed May 29, 2026, <https://ecostruxure-building-help.se.com/bms/topics/show.castle?id=5239&locale=en-US&productversion=1.8>
29. Temperature conversion — pint 0.25.3rc1.dev1+g5e79411e1 ..., accessed May 29, 2026, <https://pint.readthedocs.io/en/stable/user/nonmult.html>
30. Comprehensive Guide to Temperature Scales and Conversion - The Engineering ToolBox, accessed May 29, 2026, https://www.engineeringtoolbox.com/temperature-d_291.html
31. TEMPERATURE CONVERSION TEM P - Patriot Bolt and Fastener, accessed May 29, 2026, <https://patriotbolt.com/wp-content/uploads/2025/03/Temperature-Conversions.pdf>
32. Why does subtraction of two units.degC quantities give a units.delta_degC result but subtracting two units.kelvin gives a units.kelvin result in metpy - Stack Overflow, accessed May 29, 2026, <https://stackoverflow.com/questions/75086527/why-does-subtraction-of-two-units-degc-quantities-give-a-units-delta-degc-result>
33. Magnitudes and Other Logarithmic Units — Astropy v8.1.0.dev125+ ..., accessed May 29, 2026, https://docs.astropy.org/en/latest/units/logarithmic_units.html

34. log and exponential unit conversions · Issue #71 · hgrecco/pint, accessed May 29, 2026, <https://github.com/hgrecco/pint/issues/71>
35. A question on astropy logarithmic units behaviour - OpenAstronomy Community, accessed May 29, 2026, <https://community.openastronomy.org/t/a-question-on-astropy-logarithmic-units-behaviour/1195>
36. Logarithmic Units — pint 0.25.3rc1.dev1+g5e79411e1 documentation - Read the Docs, accessed May 29, 2026, https://pint.readthedocs.io/en/stable/user/log_units.html
37. Units and Quantities (astropy.units), accessed May 29, 2026, <https://docs.astropy.org/en/stable/units/index.html>
38. Fuel Economy Conversion | Convert MPG,& More - FloorLo, accessed May 29, 2026, <https://www.floorlo.com/fuel-economy-converter>
39. accessed May 29, 2026, <https://www.omnicalculator.com/conversion/fuel-economy-converter#:~:text=Make%20a%20note%20of%20your,of%20step%20%20by%20100.>
40. MPG to L/100 km Converter - Omni Calculator, accessed May 29, 2026, <https://www.omnicalculator.com/conversion/fuel-economy-converter>
41. MPG to l/100km Converter Miles per Gallon, accessed May 29, 2026, <https://www.mpgtolitres.com/>
42. Fuel Economy Converter — MPG to L/100km Calculator — How to Use It | Engineering Tool, accessed May 29, 2026, <https://www.youtube.com/watch?v=t23D2MXwcMw>
43. Contexts — pint 0.9 documentation, accessed May 29, 2026, <https://pint.readthedocs.io/en/0.9/contexts.html>
44. Contexts — pint 0.25.3rc1.dev1+g5e79411e1 documentation - Read the Docs, accessed May 29, 2026, <https://pint.readthedocs.io/en/stable/user/contexts.html>
45. Measurements, Units, Conversions, Density (M1Q1) – UW-Madison Chemistry 103/104 Resource Book - University of Wisconsin Pressbooks, accessed May 29, 2026, <https://wisc.pb.unizin.org/minimisgenchem/chapter/m1q1-thinking-like-a-scientist/>
46. Density Unit Converter - Alfa Chemistry, accessed May 29, 2026, <https://www.alfa-chemistry.com/resources/density-unit-converter.html>
47. Convert Units API, accessed May 29, 2026, <https://apileague.com/apis/convert-units-api/>
48. Unit Conversion API - API Ninjas, accessed May 29, 2026, <https://api-ninjas.com/api/unitconversion>
49. Open-Source-Drilling-Community/UnitConversion: .NET library for the management of unit conversions deployed as NuGet's, a microservice, and a web application - GitHub, accessed May 29, 2026, <https://github.com/Open-Source-Drilling-Community/UnitConversion>
50. Library for intelligent unit conversion? : r/Python - Reddit, accessed May 29, 2026, https://www.reddit.com/r/Python/comments/70g98f/library_for_intelligent_unit_conversion/

51. unyt: Handle, manipulate, and convert data with units in Python - arXiv, accessed May 29, 2026, <https://arxiv.org/pdf/1806.02417>
52. Pint – A Python package to define, operate and manipulate physical quantities | Hacker News, accessed May 29, 2026, <https://news.ycombinator.com/item?id=28413252>
53. Units and Unit Conversions with Python's SymPy - YouTube, accessed May 29, 2026, <https://www.youtube.com/watch?v=SE28qPzuUkM>
54. unitsofmeasurement/unit-api: Units of Measurement API - GitHub, accessed May 29, 2026, <https://github.com/unitsofmeasurement/unit-api>
55. Units of Measurement, accessed May 29, 2026, <http://unitsofmeasurement.github.io/>
56. Exploring Java's Units of Measurement API (JSR 385) - belief driven design, accessed May 29, 2026, <https://belief-driven-design.com/java-measurement-jsr-385-210f2/>
57. The Java Community Process(SM) Program - JSRs: Java Specification Requests - detail JSR# 385, accessed May 29, 2026, <https://jcp.org/en/jsr/detail?id=385>
58. accessed May 29, 2026, <https://cloud.google.com/discover/what-is-model-context-protocol#:~:text=The%20MCP%20creates%20a%20standardized,AI%20model%20and%20external%20system.>
59. Understanding How MCP Works Internally with LLMs and MCP Clients, accessed May 29, 2026, https://www.reddit.com/r/mcp/comments/1tiadzg/understanding_how_mcp_works_internally_with_llms/
60. Model Context Protocol: How One Standard Became the “USB-C for AI” | by Aftab | May, 2026, accessed May 29, 2026, <https://medium.com/@aftab001x/model-context-protocol-how-one-standard-became-the-usb-c-for-ai-08ee51f730bb>
61. What is the Model Context Protocol (MCP)? - Databricks, accessed May 29, 2026, <https://www.databricks.com/blog/what-is-model-context-protocol>
62. What is Model Context Protocol (MCP)? A guide | Google Cloud, accessed May 29, 2026, <https://cloud.google.com/discover/what-is-model-context-protocol>
63. MCP Cheat Sheet (2026) - Model Context Protocol Quick Reference | Webfuse, accessed May 29, 2026, <https://www.webfuse.com/mcp-cheat-sheet>
64. JSON-RPC in the communication layer for MCP - Reddit, accessed May 29, 2026, https://www.reddit.com/r/mcp/comments/1o99xjp/jsonrpc_in_the_communication_layer_for_mcp/
65. Best schema/prompt pattern for MCP tool descriptions? (Building an API-calling project) : r/LanguageTechnology - Reddit, accessed May 29, 2026, https://www.reddit.com/r/LanguageTechnology/comments/1rdj2tg/best_schema_prompt_pattern_for_mcp_tool/
66. Tools - Model Context Protocol (MCP), accessed May 29, 2026, <https://modelcontextprotocol.info/docs/concepts/tools/>
67. Tools - Model Context Protocol, accessed May 29, 2026, <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

68. A comprehensive Model Context Protocol (MCP) server that turns any AI assistant into a powerful mathematical computation engine. - GitHub, accessed May 29, 2026, <https://github.com/SHSharkar/MCP-Mathematics>
69. cyanheads/calculator-mcp-server - GitHub, accessed May 29, 2026, <https://github.com/cyanheads/calculator-mcp-server>
70. Introduction to Model Context Protocol - Anthropic Courses, accessed May 29, 2026, <https://anthropic.skilljar.com/introduction-to-model-context-protocol>
71. MCP Message Types: Complete MCP JSON-RPC Reference Guide - Portkey, accessed May 29, 2026, <https://portkey.ai/blog/mcp-message-types-complete-json-rpc-reference-guide/>